

Multi-Agent Orchestration with a Manager Agent

Overview

Figure: Multi-agent orchestrator pattern. A lead **manager agent** (orchestrator) can coordinate multiple specialized sub-agents in parallel to tackle a complex task ¹. For example, Anthropic's *Research* system uses a **lead agent** to analyze the user's query, devise a plan, and spawn **sub-agents** to handle different subtasks simultaneously ². Each sub-agent works on its piece (e.g. searching the web or analyzing data) with its own context window, then returns results to the manager for synthesis ³. This approach decomposes a problem into parallel streams, effectively increasing the total context and reasoning capacity beyond what a single agent could handle. A shared memory or context store is used so that the lead agent can persist plans and integrate sub-agents' findings even if they exceed a single model's context window ³. The manager agent then decides if further subtasks are needed (it can spawn more agents or refine the strategy iteratively) and finally compiles the outputs into a coherent result for the user.

Recommended Multi-Agent Architecture

Manager (Planner) Agent and Task Delegation: To put an agent "in charge" of the task, designate one agent as a **Planner/Manager** with the sole role of orchestrating others. This manager agent (ideally a powerful LLM like GPT-4 or Claude 2) receives the high-level goal, breaks it down into actionable steps, and assigns each step to a **worker agent** specialized in that type of task. For example, the manager might create subtasks like research, coding, or writing, and dispatch these to different agents: e.g. a "Researcher" agent to gather information, a "Coder" agent to handle coding tasks, a "Writer" agent to compose text, etc. Each specialized agent can operate semi-independently on its task. The manager keeps track of the overall plan and sequence – it might wait for all sub-agents to report back, then integrate their outputs, potentially loop by creating new subtasks if needed, and finally produce the final answer. This pattern was illustrated by Codanyks, where a `PlannerAgent` generates a list of subtasks and later a `BuilderAgent` (worker) picks them up and executes them ⁴. In a chain-of-thought style pipeline, one could have a Planner agent define tasks, a Researcher agent gather context, a Writer agent draft content, and a Reviewer agent provide feedback, each handing off to the next via shared context ⁵. The key is that **all coordination logic resides in the manager agent**: it uses the outputs and state of sub-agents (via shared memory or messages) to decide next steps. This loose coupling through shared context ensures agents can be added or removed without hard-coding their interactions ⁶ ⁴.

MCP Server for Offloading Tasks: To enable an agent to spawn and manage other agents, using a Model Context Protocol (MCP) server is highly effective. An MCP server acts as a broker that your manager agent can call to delegate subtasks to external processes or distinct model instances. In practice, the manager (as an MCP *client*) will send a request describing the subtask to an MCP server, which in turn invokes another AI agent or tool to handle that subtask, then returns the result. This allows the primary agent to **offload context** and computation to a sub-agent, preventing the manager's own context window from being overwhelmed ⁷. For your use case, a specialized MCP server like "**MCP Inception**" would be ideal – it's designed as "*an agent for your agent*" that lets one agent call another LLM agent to handle a query, and even supports parallel or map-reduce style task execution ⁷ ⁸. With MCP Inception, your manager can issue

an `execute_mcp_client` call with a question or task, and the MCP server will spin up a separate LLM session to solve it (ignoring intermediate chain-of-thought steps and returning only the final answer) ⁸. It also supports `execute_parallel_mcp_client` for running multiple queries in parallel (useful if the manager wants to ask the same question to multiple agents or split data for processing) and even a map-reduce pattern for aggregation ⁹ ¹⁰. In essence, this kind of MCP setup allows your “boss” agent to **spawn sub-agents on demand** and later combine their results.

If your project involves a lot of coding or code review tasks, you might also consider the **Aider MCP Server** as a specialized option. This MCP server is built to offload coding work to Aider (an open-source AI coding assistant) ¹¹. In an orchestrated workflow, your manager agent (perhaps using Anthropic’s Claude or OpenAI) could delegate “write this code” or “modify that file” tasks to Aider via the MCP server, which gives you the benefits of Aider’s code-editing prowess while keeping the high-level planning in the manager’s hands ¹¹. Offloading to Aider can reduce costs and let you leverage a dedicated coding agent that edits files, runs tests, etc., under the manager’s direction ¹¹. There are many MCP servers available (for filesystems, web browsing, etc.), but for a manager-offloads-to-agents scenario, the “**inception**” style (task delegation) MCP server is most relevant ⁷. It provides a standardized interface (using JSON-RPC over STDIO or HTTP) so your agents communicate with tools and each other in a controlled, decoupled way ¹² ¹³. In summary, yes – using an MCP server is recommended to achieve the kind of hierarchical agent setup you described, and a server designed for sub-agent delegation (like MCP Inception) would be a good choice to start with.

Long-Term Memory and Shared Context: Alongside the manager/worker agent structure, you will want a mechanism for **persistent memory** so that information can be shared among agents and remembered across steps. The manager might need to log the overall plan and partial results, and each sub-agent might produce findings that others should see. A **shared memory store** or blackboard system (which could be as simple as a database or as sophisticated as a purpose-built memory server) is often used in multi-agent frameworks ¹⁴ ¹⁵. In your setup, leveraging a long-term memory system like *MemOS* would be very valuable. *MemOS* is essentially an “operating system” for LLM memory – it allows an agent to store, retrieve, and manage knowledge beyond the immediate chat context ¹⁶. For example, when the lead agent formulates its plan, it can persist that plan (and any background info) into *MemOS*’s store. Sub-agents can query this store for relevant info or to understand the task context. *MemOS* supports multiple types of memory (textual knowledge, cached key/value, even parametric fine-tune info) in a structured way ¹⁷ ¹⁸. By integrating a memory system, your agents won’t be limited by the fixed context window – they can **recall facts or past interactions on the fly**, enabling more consistent long-running workflows. Whether you use *MemOS* or another solution, ensure the manager and sub-agents have access to a **common memory interface** to read/write important data (tasks, results, decisions). This could be done through the MCP (e.g. an MCP memory server) or via direct API calls to a database. The key is that memory should be external to the agents’ prompts, so that it persists and can be accessed by any agent as needed ¹⁹ ¹⁵. (Notably, frameworks like LangChain’s **LangGraph** also offer built-in memory modules: short-term conversational state persisted via checkpoints, and long-term stores for facts or embeddings ²⁰ ²¹. *LangGraph*’s memory can serve a similar role if you use that framework – it ensures conversation history and data survive beyond one session, and can be retrieved across agents or threads.)

Implementing the Architecture: There are a couple of ways to implement this multi-agent manager/worker system:

- *Custom Orchestrator via MCP:* One approach is to write a custom orchestrator script that uses the MCP client libraries to spawn tasks. For example, your manager agent (running in a loop in code) could detect when a subtask is needed (“Search for X”, “Code Y”, etc.), and then call an MCP server (like `mcp-inception`) to handle it. The MCP server will manage invoking another model or tool for that subtask and return the result. Your manager agent then incorporates that result into its own reasoning. This approach gives you fine control and is very flexible: you can plug in different MCP servers (web search, coding, calculation, etc.) as tools for the manager. The trade-off is you’ll need to manage the conversation prompts and parsing of results between the manager and sub-agents manually.
- *Frameworks (AutoGen or CrewAI):* Using a high-level multi-agent framework can simplify the orchestration. **AutoGen** (by Microsoft) conceptualizes multi-agent systems as conversations between agents ²². You can create agents with different roles and have them talk to each other in a structured way. For instance, you’d instantiate a **PlannerAgent** and several **AssistantAgents** (workers), and AutoGen can facilitate patterns like a **Group Chat** (all agents in one conversation with a coordinator) or **Nested chats** (an agent can internally trigger a sub-conversation for a subtask) ²³ ²⁴. AutoGen comes with a GroupChatManager that essentially plays the role of your “manager” by directing which agent should speak when and coordinating the information flow ²⁵. This can naturally implement a hierarchical workflow (the manager agent decides the order of agent interactions, ensuring, say, the Researcher speaks first, then the Writer, etc.). AutoGen also allows agents to execute code or tools as part of the conversation, which can be useful for complex tasks.
- **CrewAI** is another framework, which you are already exploring. CrewAI offers a very explicit way to define agents with roles, and **tasks** that you assign to those agents in a crew workflow ²⁶ ²⁷. In CrewAI, you would define a crew (team) that includes, say, a Planner agent and several worker agents. You then define **Task** objects for each piece of work, including which agent is responsible and what tools they can use. CrewAI supports processes (workflow patterns) that can be sequential (agents take turns in a fixed order) or even hierarchical branching ²⁸ ²³. This structured approach can be very intuitive: you literally specify “Agent A will do Task 1, then pass result to Agent B for Task 2,” etc., and CrewAI handles the messaging under the hood. The advantage here is clarity and simplicity in setup – tasks have clear descriptions and expected outputs, and the framework handles inter-agent communication. For example, defining a task might look like: *Task: Scrape a website (expected output: text content), assigned to WebScraperAgent; Task: Analyze content (expected output: summary), assigned to AnalystAgent*, etc. CrewAI will run the agents in order (or in parallel if configured) and ensure each gets the necessary context ²⁹. Under the hood, CrewAI can utilize LangChain tools and memory, meaning you can integrate things like web search, file I/O, or vector DB retrieval as part of an agent’s toolset ³⁰. It even allows agents to have **persistent memory** across turns if enabled, improving handling of multi-step tasks ³¹. In summary, frameworks like CrewAI provide a high-level API to set up your manager-worker architecture without dealing with low-level MCP calls directly – you declare the roles, tasks, and sequence, and the library takes care of the orchestration logic.

Both approaches (custom MCP-based vs. using a framework) can achieve the goal. In fact, they’re not mutually exclusive: frameworks can often be extended. For instance, AutoGen and CrewAI can themselves

call MCP tools – you could have a CrewAI agent use a custom tool that, behind the scenes, triggers an MCP server call. Or you could use **LangGraph** as an overarching layer to wire different frameworks together. Recent examples show that you can integrate AutoGen with LangGraph to get benefits like persistent memory, streaming outputs, and even deploy the multi-agent system easily ³². LangGraph essentially can act as the glue or *workflow manager*, where one node in its graph might invoke an AutoGen conversation or a CrewAI process, and subsequent nodes handle memory saving or UI display. This means you could prototype with CrewAI + MemOS now, and later slot in AutoGen (or another agent framework) by adjusting the graph, without losing capabilities like memory and persistence. The ecosystem is evolving rapidly, so choose the tools you’re most comfortable with – the core architecture (manager + sub-agents + memory) remains the same.

Implementing with CrewAI, MemOS, and LangGraph (Your Planned Setup)

Given the components you’ve started with – **CrewAI** for multi-agent orchestration and **Memory Tensor’s MemOS** for memory – we can outline how to realize the manager-agent architecture using those (and note how LangGraph or AutoGen might come into play down the road):

- **CrewAI as the Orchestrator:** In CrewAI, begin by defining your agents and their roles. You will want one agent to be the **Coordinator** (the “boss” agent). This agent’s role could be something like *Project Manager* with a goal of “break down tasks and delegate to the team.” The other agents will have specialized roles (e.g. *Researcher*, *Coder*, *Writer*, etc., depending on your needs). Each agent in CrewAI can have its own LLM (you might use a powerful model for the manager and smaller models for workers to save cost, or the same model with different system prompts for persona) ³³. Next, define the **Tasks** in CrewAI’s terminology. For a hierarchical workflow, you might use CrewAI’s *sequential process* where the manager agent executes first, then triggers others. Concretely: you could create a Task for the Manager agent like “Plan the solution and list subtasks,” and then Tasks for each sub-agent like “Perform subtask X” with the `assigned_to` field set to the respective agent ³⁴. The manager’s output (a plan or list of subtasks) can be fed into the next tasks. CrewAI will manage passing the context – the framework ensures the content produced by one agent can be provided as input to the next agent in the crew sequence ²⁹. This is essentially your manager agent *offloading* work: the manager’s task produces a plan, and CrewAI then hands each part of that plan to the appropriate agent in turn. If you need more dynamic delegation (not a fixed sequence), CrewAI also supports a **hierarchical process** (one agent can spawn subtasks under the hood). You might need to implement custom logic in the manager agent’s prompt (or use CrewAI’s `Process` API) to conditionally invoke agents. But given CrewAI’s flexibility, a straightforward way is to use a sequential process where the first agent is the planner and subsequent ones execute subtasks in order. After all tasks are done, you can even have a final task where the manager agent reviews all results and composes the final answer (essentially coming back to the manager for the conclusion).
- **Integrating MemOS for Memory:** CrewAI provides a way to maintain agent memory across turns (by enabling the `memory` option for agents), but MemOS can greatly extend this by giving a **long-term, content-addressable memory** store for all agents. You can run MemOS as a service or use it via its Python API within your CrewAI agents. One pattern could be: at the start of the workflow, have the manager agent query MemOS for any relevant prior knowledge (for example, if the task is similar to something done before, the manager can pull those notes). During execution, agents can

call MemOS to **store intermediate results or key information** that might be needed later. For instance, after a Researcher agent finds some data, it could save those findings into MemOS so that the Writer agent later can retrieve them. MemOS's API allows adding memory entries (tagged by a user or task ID) and searching for them by relevance ³⁵ ³⁶. To hook this into CrewAI, you could create a simple **tool** or utility that the agents can use (e.g., a `MemorySearchTool` and `MemoryAddTool`). Since CrewAI supports custom tools (and even integrates with LangChain's tool library), your agents can call something like `memory_search("query")` to fetch from MemOS and `memory_add("content")` to save new info. This effectively gives your agent team a shared "wiki" or scratchpad throughout the project. By the end of a run, the MemOS store will contain a trace of what was done, which could be persisted to disk for the next session. Long-term, MemOS enables more **context-aware and consistent behavior** – agents can remember user preferences, facts discovered earlier, or decisions made, even as you offload tasks or exceed normal token limits ¹⁶ ¹⁷.

- **Using LangGraph (optional):** You mentioned running MemOS with LangGraph simultaneously; this suggests you might be leveraging LangGraph for its UI or persistence. If you deploy your solution on LangGraph, you could wrap the CrewAI workflow in a LangGraph node as a function call. For example, you define a LangGraph `StateGraph` where one step calls a Python function that executes your CrewAI "crew.run()" process and returns the result. LangGraph would allow you to **checkpoint** the conversation state around that (so you can reconnect to the same session later), and you could use its built-in memory stores as well, alongside MemOS. In fact, LangGraph's documentation shows how one can integrate other agent frameworks by treating them as black-box calls within the graph ³². This means you could start with CrewAI now, and later swap in AutoGen or another approach with minimal changes to the overall architecture – the graph orchestrates at a higher level. Additionally, LangGraph has a *Multi-agent* and *MCP* integration in its roadmap (as seen in its docs navigation ³⁷), so it is very much aligned with these kinds of setups. The bottom line: LangGraph is not required, but if you already have it, it can provide a convenient way to visualize and manage the interactions, and ensure nothing is lost between runs (with its database-backed memory for chat threads ²⁰). It can also handle front-end concerns (like a chat interface or API endpoint for your system) so you can focus on the agent logic.
- **Framework Transitions (CrewAI vs AutoGen):** Since you mentioned possibly moving to AutoGen in the future, design your system in a modular way. Right now, CrewAI + MemOS can achieve the goal. In the future, you might use AutoGen's group chats to let agents converse freely. The high-level roles would remain: you'd still have a planner (which could be an AutoGen agent that perhaps uses a GroupChatManager or a "System" role to guide others) and workers. AutoGen might not use "tasks" objects like CrewAI, but you can enforce structure through the conversation patterns (e.g., the planner agent always speaks first to outline the plan, then the others respond accordingly). One benefit of AutoGen is that it was built with the idea of agents reasoning **collaboratively** in natural language, which sometimes can surface creative solutions. CrewAI, being more structured, makes it easier to ensure the correct order and use of tools. You might find CrewAI sufficient for now, and when switching to AutoGen, just be mindful to incorporate a mechanism for long-term memory (which you can still do by calls to MemOS or using LangGraph's stores). AutoGen doesn't magically persist information by itself, so you'd want to plug in a memory just as you are doing now. The fact that LangGraph can work with both means you won't lose your investment in memory and data handling when you change the agent orchestration framework.

Using Continue.dev and Its Tools (Optional)

You also asked whether to run this setup with **Continue.dev** and its toolset. **Continue** is an open-source AI coding assistant that integrates into IDEs (like VS Code or JetBrains) ³⁸. It's mainly designed to help with code autocompletion, editing, and querying your codebase using LLMs, acting somewhat like GitHub Copilot but with more openness and customizability. Continue.dev isn't specifically a multi-agent orchestration framework; rather, it's an environment where an AI agent helps a developer write or modify code. That said, Continue is quite extensible – it supports custom “blocks” (plugins) for different contexts and even MCP servers ³⁹. For instance, Continue can be configured with an **MCP block** which means it can communicate with MCP servers as part of its operation ³⁹. In theory, you could incorporate your multi-agent system into Continue so that your IDE assistant (Continue) delegates tasks to your other agents. For example, if you're working in VS Code and using Continue, you might have it set up so that when you ask a high-level question, Continue's AI (say GPT-4) acts as the manager and, via an MCP call, engages your CrewAI system or a specific MCP server to handle a subtask (like running some code or doing a web search), then returns the result in the IDE chat.

However, using Continue.dev is **not a requirement** for running your architecture – it's more about your development workflow. You can absolutely run the CrewAI+MemOS system as a standalone script or server, independent of Continue. Continue could be helpful during **development and testing**: for example, you might use it to quickly prototype agent prompts or to call your own system via an MCP interface while you sit in the IDE. It can also leverage all its built-in tools (like codebase search, git integration, etc.) which might overlap with what some of your agents do. If one of your agents' tasks is code editing, sometimes just letting Continue's single-agent do it interactively with you might suffice. But if the goal is full automation with multiple agents, you won't gain much by *only* running it inside Continue – in fact, it could complicate things to have the extra layer.

A sensible approach is: **use Continue.dev if it adds value for you**, such as:

- *During development*: to assist you in writing the code for your agents, setting up MCP config, or quickly testing queries. Continue can speed up coding with its AI suggestions and allow you to chat with an AI about your codebase (which might include your CrewAI scripts) – this can be a productivity boost but doesn't change the architecture itself.
- *For leveraging IDE tools*: If your multi-agent system involves tasks like reading/writing files, running code, or git operations, you have a choice: either implement those as agent tools (possibly via Python or shell commands), or use an environment like Continue which already has some of those capabilities. For example, Continue (and similar IDE agents) can read the repository files, modify code, etc. If you find it easier, you could let the manager agent's “coding” subtasks be handled by Continue's agent (which is essentially what the Aider MCP server does with Claude+Aider). This would mean hooking up your manager to communicate with Continue's backend – not trivial, but possible via MCP or an API.

In summary, **Continue.dev is not strictly necessary** for a multi-agent setup – frameworks like CrewAI/AutoGen will run on their own. But you *can* integrate it if you want an interactive development loop or to utilize its pre-built tool integrations. Since Continue puts you “in control” of custom AI assistants, you could configure a custom assistant in Continue that knows about your CrewAI or calls into your system. Just keep in mind that Continue is focused on assisting a human developer; if your aim is to fully automate agents talking to each other, you might keep that logic outside the IDE and perhaps use Continue as a **monitor or**

helper. For instance, you could run your multi-agent system in a terminal, and use Continue in VS Code to inspect logs or memory, or to manually intervene if an agent gets stuck.

To directly answer your question: you **can** run your setup with Continue and its tools, but it's **optional**. If you do, consider using the MCP integration in Continue (the "MCP blocks") to connect your orchestrator – this would allow Continue's AI to invoke your MCP servers or vice versa ³⁹. One could imagine the manager agent living in a headless service, and Continue's interface just controlling or kicking off the process. Many developers simply use Continue for building the system, then deploy the system separately. So feel free to use it as a development aid or not at all. Your multi-agent architecture will work regardless, as long as the pieces (manager agent, sub-agents, memory store, MCP server) are properly connected.

Key Takeaways: You can achieve a powerful multi-agent system by having one agent act as the **project manager** that delegates to others. Using an MCP server geared for task delegation will make it easier to spawn and coordinate sub-agents ⁴⁰. Frameworks like CrewAI (structured tasks/workflows) and AutoGen (conversational agent collaboration) can abstract a lot of the orchestration for you – they are both viable, with CrewAI offering more explicit control of task flow and AutoGen offering flexible agent communication patterns ⁴¹ ⁴². Your current plan of using CrewAI with MemOS for long-term memory is solid: CrewAI will let you define the team structure and task sequence, while MemOS ensures the agents share context and remember facts beyond the context window ¹⁶. Integrating LangGraph can further provide persistence and a path to scale or visualize the system. And tools like Continue.dev or Aider are there to help with coding work, but they are **adjuncts** to the core architecture. It's great that you're exploring these cutting-edge components – with a clear architecture in mind (manager agent + MCP + memory + specialized workers), you can mix and match implementations. Over time, you might refine prompts, add more MCP servers (for example, one for web browsing, one for database queries, etc.), or swap out the LLM models as needed. The architecture is modular, so keep the interfaces clean (e.g., how the manager defines tasks and how workers report back) and you'll be able to evolve the system. Good luck with building your multi-agent setup!

Sources:

- Anthropic, "*How we built our multi-agent research system*" – Orchestrator-worker agent pattern and memory usage ¹ ³.
- Codanyks, "*Agent-to-Agent Communication via MCP*" – Describes using MCP as a shared context for agents to delegate tasks asynchronously ⁴ ⁵.
- Tane Van Wifferen, "*mcp-inception*" – An MCP server enabling an agent to delegate queries to another LLM (sub-agent), supporting parallel and map-reduce calls ⁷ ⁸.
- Disler (GitHub), "*Aider MCP Server – Overview*" – Offloading coding tasks to Aider via MCP, allowing a lead agent (Claude) to delegate code edits for efficiency ¹¹.
- CrewAI Documentation – Multi-agent framework with agents, tasks, crews, and memory integration ²⁹ ³¹.
- Christian Spano, "*CrewAI vs. AutoGen*" – Comparison of frameworks; CrewAI uses explicit tasks and roles, while AutoGen coordinates agents via conversations (group chats, nested chats) ³⁴ ²³.
- MemTensor MemOS – Memory OS for LLMs providing long-term memory storage and retrieval for more context-aware agents ¹⁶.
- LangChain LangGraph – Guide on integrating AutoGen with LangGraph for persistence, memory, multi-agent orchestration ³².

- Continue.dev – Open-source IDE AI assistant (supports custom tools and MCP integration) 38 39 .

1 2 3 How we built our multi-agent research system \ Anthropic

<https://www.anthropic.com/engineering/built-multi-agent-research-system>

4 5 6 14 15 19 Agent-to-Agent Communication with MCP Server | Codanyks | Medium

<https://medium.com/@codanyks/agent-to-agent-communication-via-mcp-30b105570d00>

7 8 9 10 40 mcp-inception MCP Server | MCP Servers · LobeHub

<https://lobehub.com/mcp/tanevanwifferen-mcp-inception>

11 GitHub - disler/aider-mcp-server: Minimal MCP Server for Aider

<https://github.com/disler/aider-mcp-server>

12 13 Agentic MCP and A2A Architecture: A Comprehensive Guide | by Anil Jain | AI / ML Architect | Data Architect | Medium

<https://medium.com/@anil.jain.baba/agentic-mcp-and-a2a-architecture-a-comprehensive-guide-0ddf4359e152>

16 17 18 35 36 GitHub - MemTensor/MemOS: MemOS (Preview) | Intelligence Begins with Memory

<https://github.com/MemTensor/MemOS>

20 21 37 Overview

<https://langchain-ai.github.io/langgraph/concepts/memory/>

22 23 24 25 28 34 41 42 CrewAI vs. AutoGen: From Basic to Complex Workflows Design | by Christian Spano | Data Reply IT | DataTech | Medium

<https://medium.com/data-reply-it-datatech/crewai-vs-autogen-from-basic-to-complex-workflows-design-39917a06af88>

26 27 29 30 Building a multi agent system using CrewAI | by Vishnu Sivan | The Pythoneers | Medium

<https://medium.com/pythoneers/building-a-multi-agent-system-using-crewai-a7305450253e>

31 Agents - CrewAI

<https://docs.crewai.com/en/concepts/agents>

32 Deploy CrewAI, AutoGen, and other frameworks

<https://langchain-ai.github.io/langgraph/how-tos/autogen-integration/>

33 LLMs - CrewAI

<https://docs.crewai.com/en/concepts/llms>

38 39 Continue

<https://www.continue.dev/>